

COMPREHENSIVE PROJECT SUMMARY

Internship Project: CRM & Lead Management System

TABLE OF CONTENTS

- [1. Project Description](#)
- [2. Tech Stack](#)
- [3. Architecture Overview](#)
- [4. Key Features](#)
- [5. Data Flow](#)
- [6. Database Schema](#)
- [7. Frontend Structure](#)
- [8. Backend Structure](#)
- [9. API Endpoints](#)
- [10. Key Services & Logic](#)
- [11. Environment Variables](#)
- [12. Deployment & Running](#)
- [13. Architectural Decisions](#)

PROJECT DESCRIPTION

This is a **full-stack CRM & Lead Management System** designed as an internship project. It's a robust, modern web application that automates lead ingestion from LinkedIn, performs AI-powered OCR document scanning for resume/business card processing, and provides real-time dashboard updates for administrators. The system eliminates manual data entry by integrating webhooks, intelligent document parsing, and Server-Sent Events (SSE) for live notifications.

Key Objectives:

- Automated lead capture from LinkedIn Lead Gen Forms via webhooks
- AI-powered resume and business card scanning using Google Gemini API
- Real-time dashboard updates for administrators using Server-Sent Events (SSE)
- Manual lead entry and verification workflows
- Bill/invoice scanning and annotation capabilities
- Complete authentication and authorization system

TECH STACK

Frontend

- Framework:** Angular 17 (Latest LTS)
- Language:** TypeScript 5.4
- Build Tool:** Angular CLI 17.3
- HTTP Client:** Angular's HttpClientModule
- Forms:** ReactiveFormsModule & FormsModule (reactive & template-driven)
- PDF Handling:** pdfjs-dist 3.11
- Reactive Programming:** RxJS 7.8
- Testing:** Jasmine, Karma, Chrome Launcher
- Styling:** Custom dark-theme CSS with responsive grid layouts

Backend

- **Runtime:** Node.js
- **Framework:** Express.js 4.19 (REST API)
- **Database Driver:** pg (PostgreSQL native driver)
- **File Upload:** Multer 2.1 (multipart/form-data handling)
- **AI Integration:** Google Generative AI SDK (@google/generative-ai)
 - Model: Gemini 2.5 Flash
- **Middleware:** CORS, JSON body parser
- **Development:** Nodemon (auto-reload on file changes)
- **Environment Management:** Dotenv

Database

- **DBMS:** PostgreSQL
- **Connection Pooling:** Via pg library
- **Tables:**
 - leads (primary CRM table)
 - notifications (system notifications)
 - profile (user profiles)
 - ocr_templates (OCR configuration)

External Services

- **LinkedIn:** Webhook integration for automated lead capture
- **Google Gemini API:** AI-powered document extraction and parsing

ARCHITECTURE OVERVIEW

Visual Architecture



High-Level Data Flow

The system follows a **three-tier architecture**:

1. Presentation Tier (Angular Frontend)

- User interface components
- Real-time SSE listeners
- Form handling and validation

- PDF viewing and annotation

2. Application Tier (Node.js/Express Backend)

- RESTful API endpoints
- Business logic
- AI integration layer
- Webhook processing
- Real-time event broadcasting

3. Data Tier (PostgreSQL Database)

- Persistent data storage
 - Relationship management
 - Transaction support
-

KEY FEATURES

1. Real-Time Lead Dashboard

- **SSE (Server-Sent Events)** stream for live updates
- Instant notification when new leads arrive
- No page refresh required
- Multi-client broadcast (all logged-in admins see updates simultaneously)
- Full CRUD operations (Create, Read, Update, Delete)
- Lead filtering and sorting capabilities
- Status tracking and management

2. LinkedIn Webhook Integration

- Automatic lead capture from LinkedIn Lead Gen Forms
- Challenge/Response verification protocol for security
- Deep parsing of nested `formResponse.answers` payload
- Automatic duplicate detection by LinkedIn ID
- Data extraction capabilities:
 - Full Name
 - Email Address
 - Phone Number
 - Company Name
- Direct database insertion upon verification

3. AI-Powered OCR Scanner

- **Drag & drop interface** for resumes & business cards
- Supported file formats: PDF, JPG, PNG
- **Google Gemini 2.5 Flash** AI model integration
- Auto-extracts structured data:
 - Full Name
 - Email Address
 - Phone Number
 - University/College
 - Degree
 - Skills (comma-separated)
- Graceful handling of missing fields ("Not found in document")
- Auto-population of form fields on extraction
- Base64 encoding for API transmission

4. Bill Scanner

- Invoice & receipt processing

- Document annotation capabilities
- PDF viewer integration with pdfjs-dist
- Data extraction from financial documents
- Support for various invoice formats

5. Annotation Tool

- Interactive PDF markup and highlighting
- Drawing tools for document annotation
- Save and persist annotations
- Collaborative annotation support

6. Authentication & Authorization

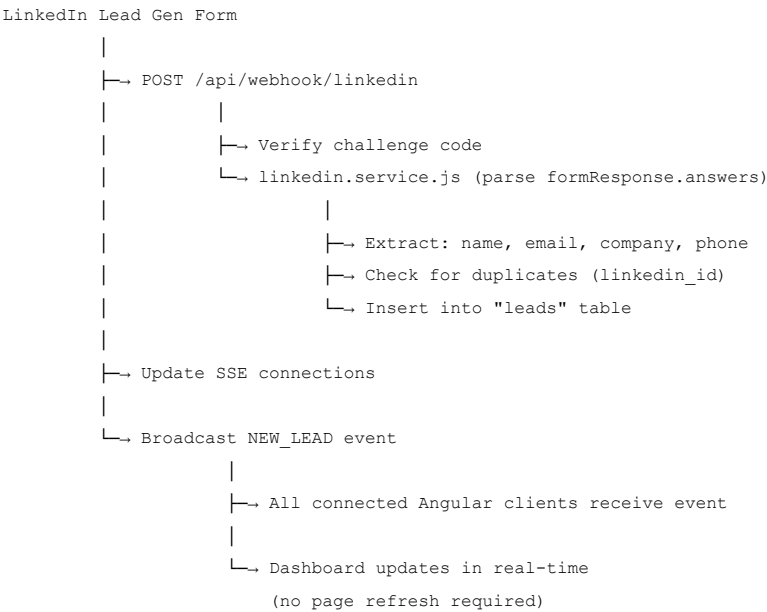
- Admin login system with credential validation
- Session-based authentication
- Protected routes and API endpoints
- User profile management
- Secure password handling

7. Manual Lead Entry

- Form-based lead creation
- Data validation and error handling
- Real-time verification workflow
- Support for multiple data sources

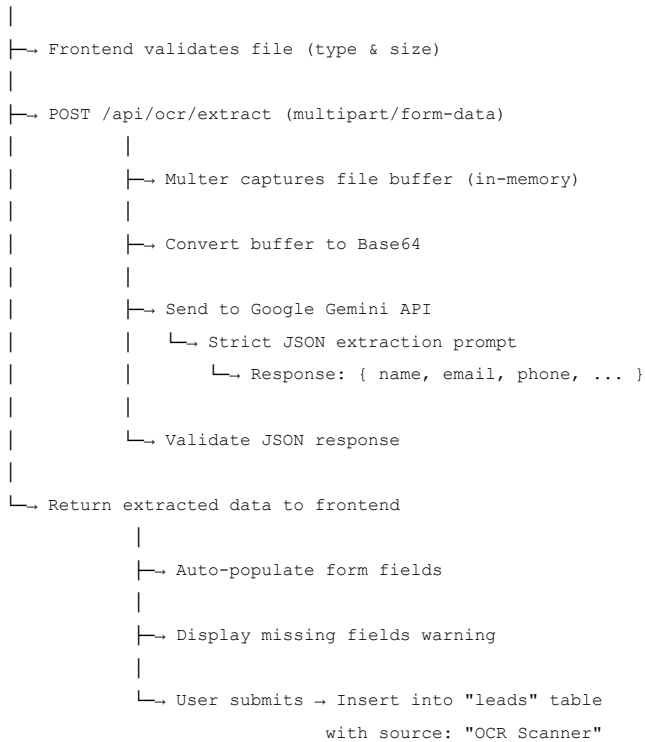
DATA FLOW

LinkedIn Lead Ingestion Flow



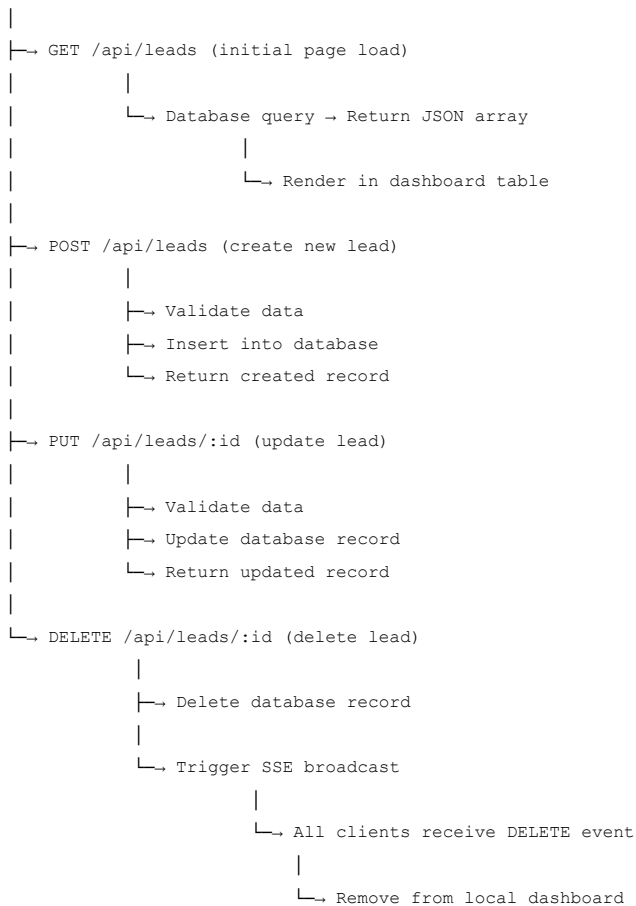
OCR Document Processing Flow

User Uploads Resume/Card

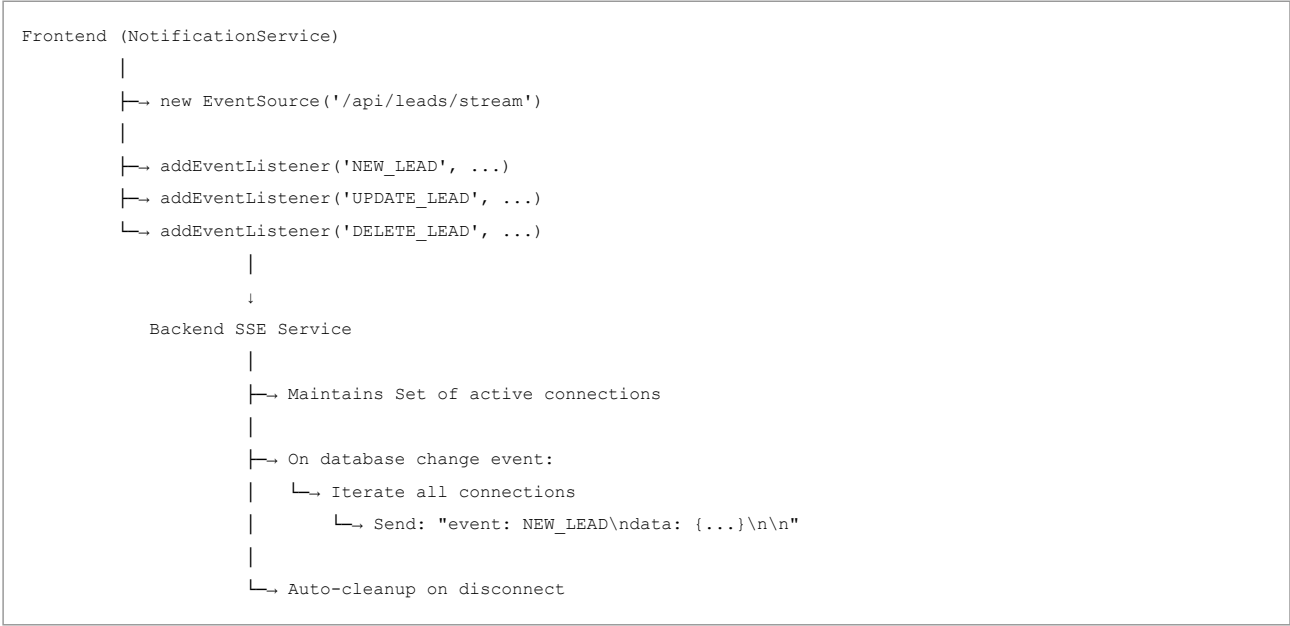


Real-Time Dashboard Update Flow

Admin Dashboard



SSE Stream Connection Flow



DATABASE SCHEMA

Core Tables

1. leads Table (Primary CRM Data)

Column	Type	Constraints	Description
id	SERIAL	PRIMARY KEY	Unique identifier, auto-increment
name	VARCHAR(255)	NOT NULL	Lead/prospect full name
email	VARCHAR(255)	UNIQUE, NOT NULL	Contact email address
phone	VARCHAR(20)	NULLABLE	Contact phone number
department	VARCHAR(100)	NULLABLE	Department/job title
status	VARCHAR(50)	DEFAULT: 'New'	Lead status (New, Contacted, Qualified, Converted, Lost)
source	VARCHAR(100)	NOT NULL	Origin: 'LinkedIn Lead Gen', 'OCR Scanner', 'Manual Entry', 'Bill Scanner'
company	VARCHAR(255)	NULLABLE	Organization/company name
notes	TEXT	NULLABLE	Extended data (University, Degree, Skills stored as JSON string)
linkedin_id	VARCHAR(255)	UNIQUE, NULLABLE	LinkedIn profile identifier for duplicate detection
created_at	TIMESTAMP	DEFAULT: CURRENT_TIMESTAMP	Record creation timestamp
updated_at	TIMESTAMP	DEFAULT: CURRENT_TIMESTAMP	Last modification timestamp

Sample Record:

```
{
  "id": 1,
  "name": "John Doe",
  "email": "john@example.com",
  "phone": "+1-555-0123",
  "department": "Software Engineer",
  "status": "Qualified",
  "source": "OCR Scanner",
  "company": "Acme Corp",
  "notes": "{\"university\": \"MIT\", \"degree\": \"BS Computer Science\", \"skills\": \"Python, JavaScript, React\"}",
  "linkedin_id": "12345678",
  "created_at": "2026-05-27T10:30:00Z",
  "updated_at": "2026-05-27T10:30:00Z"
}
```

2. notifications Table (Event Tracking)

Column	Type	Description
id	SERIAL PRIMARY KEY	Unique notification ID
event_type	VARCHAR(50)	Event type: NEW_LEAD, UPDATE_LEAD, DELETE_LEAD
payload	JSONB	Complete event data
timestamp	TIMESTAMP	When event occurred
read	BOOLEAN	Whether notification was read by user

3. profile Table (User Profiles)

Column	Type	Description
id	SERIAL PRIMARY KEY	User ID
username	VARCHAR(100) UNIQUE	Admin username
email	VARCHAR(255) UNIQUE	Admin email
settings	JSONB	User preferences and settings
created_at	TIMESTAMP	Account creation date
last_login	TIMESTAMP	Last login timestamp

4. ocr_templates Table (Configuration)

Column	Type	Description
id	SERIAL PRIMARY KEY	Template ID
name	VARCHAR(255)	Template name
document_type	VARCHAR(100)	Resume, Business Card, Invoice, etc.
field_mappings	JSONB	Field extraction configuration
active	BOOLEAN	Whether template is active

FRONTEND STRUCTURE

Directory Layout


```

Frontend/src/app/
├─ app.module.ts                # Root module declaration
├─ app-routing.module.ts        # Route configuration
├─ app-state.service.ts         # Centralized state management
├─ app.component.ts             # Root component
├─
├─ login/                       # Authentication module
│   ├─ login.component.ts
│   └─ login.component.spec.ts
├─
├─ main-layout/                 # Main layout container
│   ├─ main-layout.component.ts
│   ├─ main-layout.component.spec.ts
│   └─ menu/                    # Navigation sidebar
│       └─ menu.component.ts
├─
├─ dashboard/                   # Real-time lead dashboard
│   ├─ dashboard.component.ts    # Main lead listing & SSE listener
│   ├─ dashboard.component.spec.ts
│   └─ (SSE updates, CRUD UI)
├─
├─ add-employee/                # Manual lead entry form
│   ├─ add-employee.component.ts
│   └─ add-employee.component.spec.ts
├─
├─ verify-employee/             # Lead verification workflow
│   ├─ verify-employee.component.ts
│   └─ verify-employee.component.spec.ts
├─
├─ ocr-scanner/                 # AI Resume/Card scanner
│   ├─ ocr-scanner.component.ts  # Drag-drop, AI extraction
│   ├─ ocr-scanner.component.spec.ts
│   └─ (Google Gemini integration)
├─
├─ bill-scanner/                # Invoice scanner module
│   ├─ bill-scanner.component.ts  # Bill upload & processing
│   ├─ bill-scanner.component.spec.ts
│   ├─ bill-scanner.service.ts    # Invoice extraction logic
│   └─ bill-scanner.service.spec.ts
├─
├─ annotation-tool/             # PDF markup & annotation
│   ├─ annotation-tool.component.ts # PDF viewer with drawing
│   └─ annotation-tool.component.spec.ts
├─
└─ Services:
    ├─ lead.service.ts           # HTTP calls to /api/leads
    │   ├─ getLeads()
    │   ├─ createLead(data)
    │   ├─ updateLead(id, data)
    │   └─ deleteLead(id)
    │
    ├─ notification.service.ts    # SSE stream (EventSource)
    │   ├─ connect()
    │   ├─ listen(eventType)
    │   └─ disconnect()
    │
    └─ bill-scanner.service.ts    # Invoice processing
        └─ extractBillData()

```

Module Declarations (app.module.ts)

```
@NgModule({
  declarations: [
    AppComponent,
    LoginComponent,
    DashboardComponent,
    MenuComponent,
    AddEmployeeComponent,
    VerifyEmployeeComponent,
    MainLayoutComponent,
    OcrScannerComponent,
    BillScannerComponent,
    AnnotationToolComponent
  ],
  imports: [
    BrowserModule,
    AppRoutingModule,
    HttpClientModule,          // For HTTP requests
    FormsModule,               // Template-driven forms
    ReactiveFormsModule        // Reactive forms
  ],
  providers: [],
  bootstrap: [AppComponent]
})
```

Key Services

lead.service.ts — REST API Communication

```
class LeadService {
  // Fetch all leads
  getLeads(): Observable<Lead[]>

  // Fetch single lead
  getLead(id: number): Observable<Lead>

  // Create new lead
  createLead(leadData: Partial<Lead>): Observable<Lead>

  // Update existing lead
  updateLead(id: number, updates: Partial<Lead>): Observable<Lead>

  // Delete lead
  deleteLead(id: number): Observable<void>
}
```

notification.service.ts — Server-Sent Events

```
class NotificationService {
  // Establish SSE connection to /api/leads/stream
  connect(): void

  // Subscribe to specific event type
  listen(eventType: string): Observable<any>

  // Close connection
  disconnect(): void

  // Auto-reconnect on disconnect
  private reconnect(): void
}
```

bill-scanner.service.ts — Invoice Processing

```
class BillScannerService {
  // Extract data from invoice/receipt
  extractBillData(file: File): Observable<ExtractedBillData>

  // Validate extracted data
  validateData(data: any): boolean
}
```

Routing Configuration (app-routing.module.ts)

```
const routes: Routes = [
  { path: '', redirectTo: '/login', pathMatch: 'full' },
  { path: 'login', component: LoginComponent },
  {
    path: '',
    component: MainLayoutComponent,
    children: [
      { path: 'dashboard', component: DashboardComponent },
      { path: 'add-lead', component: AddEmployeeComponent },
      { path: 'verify-lead', component: VerifyEmployeeComponent },
      { path: 'ocr-scanner', component: OcrScannerComponent },
      { path: 'bill-scanner', component: BillScannerComponent },
      { path: 'annotation', component: AnnotationToolComponent }
    ]
  }
];
```

BACKEND STRUCTURE

Directory Layout

```

Backend/
├─ index.js                # Server entry point & setup
├─ db.js                   # PostgreSQL connection pool
├─
├─ routes/                 # API endpoint handlers
│   ├─ auth.js             # Authentication endpoints
│   ├─ leads.js            # Lead CRUD + SSE stream
│   ├─ webhooks.js         # LinkedIn webhook handler
│   ├─ ocr.js              # OCR extraction endpoint
│   ├─ bill-scanner.js     # Invoice processing
│   ├─ notifications.js    # Notification endpoints
│   └─ profile.js         # User profile CRUD
├─
├─ services/               # Business logic & utilities
│   ├─ sse.service.js      # Real-time event broadcasting
│   └─ linkedin.service.js # Webhook payload parsing
├─
├─ migrations/             # Database schema setup
│   ├─ alter_db_ocr_templates.js # OCR template table creation
│   └─ alter_db_notifications.js # Notification table creation
├─
├─ .env                    # Environment configuration
├─ package.json            # Dependencies
└─ package-lock.json      # Dependency lock file

```

Server Entry Point (index.js)

```

// Load environment variables
require('dotenv').config();

const express = require('express');
const cors = require('cors');

// Import routes
const authRoutes = require('./routes/auth');
const leadsRoutes = require('./routes/leads');
const webhooksRoutes = require('./routes/webhooks');
const ocrRoutes = require('./routes/ocr');
const billScannerRoutes = require('./routes/bill-scanner');
const notificationsRoutes = require('./routes/notifications');
const profileRoutes = require('./routes/profile');

// Initialize app
const app = express();

// Middleware
app.use(cors()); // Enable CORS for all routes
app.use(express.json()); // Parse JSON bodies

// Register routes
app.use('/api/auth', authRoutes);
app.use('/api/leads', leadsRoutes);
app.use('/api/webhook', webhooksRoutes);
app.use('/api/ocr', ocrRoutes);
app.use('/api/bill-scan', billScannerRoutes);
app.use('/api/notifications', notificationsRoutes);
app.use('/api/profile', profileRoutes);

// Backward compatibility
app.use('/api/login', authRoutes);

// Database initialization
const createOcrTemplatesTable = require('./alter_db_ocr_templates');
const createNotificationsAndProfileTables = require('./alter_db_notifications');

const initializeDatabase = async () => {
  await createOcrTemplatesTable();
  await createNotificationsAndProfileTables();
};

initializeDatabase();

// Start server
const PORT = process.env.PORT || 3000;
app.listen(PORT, () => {
  console.log(`Backend running on http://localhost:${PORT}`);
});

```

Database Connection (db.js)

```
const { Pool } = require('pg');

const pool = new Pool({
  host: process.env.DB_HOST,
  port: process.env.DB_PORT,
  database: process.env.DB_NAME,
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
});

// Connection testing
pool.on('connect', () => {
  console.log('Database connected');
});

pool.on('error', (err) => {
  console.error('Unexpected error on idle client', err);
});

module.exports = pool;
```

API ENDPOINTS

Authentication Routes

Method	Endpoint	Description	Request Body	Response
POST	/api/auth/login	Admin login	{ username, password }	{ token, user }
POST	/api/auth/logout	Logout	-	{ success: true }

Leads CRUD Routes

Method	Endpoint	Description	Request Body	Response
GET	/api/leads	Fetch all leads	-	Lead[]
GET	/api/leads/:id	Fetch single lead	-	Lead
POST	/api/leads	Create new lead	Partial<Lead>	Lead
PUT	/api/leads/:id	Update lead	Partial<Lead>	Lead
DELETE	/api/leads/:id	Delete lead	-	{ success: true }
GET	/api/leads/stream	SSE stream (real-time)	-	Server-Sent Events

Webhook Routes

Method	Endpoint	Description	Request Body	Response
GET	/api/webhook/linkedin	Challenge verification	{ challenge }	{ challenge }
POST	/api/webhook/linkedin	Process lead data	LinkedIn payload	{ success: true, lead }

OCR Routes

Method	Endpoint	Description	Request Body	Response
POST	/api/ocr/extract	Extract data from document	FormData (file)	{ name, email, phone, university, degree, skills }

Bill Scanner Routes

Method	Endpoint	Description	Request Body	Response
POST	/api/bill-scan/extract	Extract invoice data	FormData (file)	{ extractedData }

Notification Routes

Method	Endpoint	Description	Request Body	Response
GET	/api/notifications	Fetch notifications -		Notification[]
POST	/api/notifications	Create notification	{ event_type, payload }	Notification
PUT	/api/notifications/:id	Mark as read	{ read: true }	Notification

Profile Routes

Method	Endpoint	Description	Request Body	Response
GET	/api/profile	Fetch user profile -		Profile
PUT	/api/profile	Update profile	Partial<Profile>	Profile
GET	/api/profile/settings	Fetch settings	-	Settings
PUT	/api/profile/settings	Update settings	Partial<Settings>	Settings

Response Status Codes

Code	Meaning
200	OK - Request successful
201	Created - Resource created successfully
204	No Content - Request successful, no content returned
400	Bad Request - Invalid input
401	Unauthorized - Authentication required
403	Forbidden - Authorization failed
404	Not Found - Resource doesn't exist
500	Internal Server Error - Server error

KEY SERVICES & LOGIC

1. SSE Service (sse.service.js)

Purpose: Manages real-time bidirectional communication with connected clients.

Functionality:

- Maintains a `Set` of active client connections
- Broadcasts events to all connected clients when database changes occur
- Handles client disconnections and cleanup
- Supports multiple event types (`NEW_LEAD`, `UPDATE_LEAD`, `DELETE_LEAD`)

Implementation:

```

class SSEService {
  constructor() {
    this.clients = new Set(); // Active connections
  }

  // Add new client connection
  addClient(response) {
    this.clients.add(response);
  }

  // Remove client connection
  removeClient(response) {
    this.clients.delete(response);
  }

  // Broadcast event to all clients
  broadcast(eventType, data) {
    const event = `event: ${eventType}\n`;
    const payload = `data: ${JSON.stringify(data)}\n\n`;

    this.clients.forEach(client => {
      if (!client.headersSent) {
        client.write(event + payload);
      }
    });
  }

  // Handle client disconnect
  setupClientCleanup(request, response) {
    request.on('close', () => {
      this.removeClient(response);
    });
  }
}

module.exports = new SSEService();

```

Usage in Routes:

```

// In leads.js route
app.get('/stream', (req, res) => {
  res.setHeader('Content-Type', 'text/event-stream');
  res.setHeader('Cache-Control', 'no-cache');
  res.setHeader('Connection', 'keep-alive');

  sseService.addClient(res);
  sseService.setupClientCleanup(req, res);
});

```

2. LinkedIn Service (linkedin.service.js)

Purpose: Parses complex LinkedIn webhook payload and extracts user information.

LinkedIn Payload Structure:


```
{
  "formResponse": {
    "submittedAt": "2026-05-27T10:30:00Z",
    "answers": [
      { "questionId": "1001", "text": "John Doe" },
      { "questionId": "1002", "text": "john@example.com" },
      { "questionId": "1003", "text": "+1-555-0123" },
      { "questionId": "1004", "text": "Acme Corp" }
    ]
  }
}
```

Implementation:

```

class LinkedInService {
  // Parse webhook payload and extract key fields
  parseFormResponse(payload) {
    const answers = payload.formResponse.answers;

    const data = {
      name: this.extractField(answers, 'name'),
      email: this.extractField(answers, 'email'),
      phone: this.extractField(answers, 'phone'),
      company: this.extractField(answers, 'company'),
      linkedin_id: payload.leadGenFormId || null,
      source: 'LinkedIn Lead Gen',
      status: 'New'
    };

    return data;
  }

  // Helper: Extract field from answers array
  extractField(answers, fieldType) {
    // Field mapping based on LinkedIn question IDs
    const mapping = {
      'name': ['1001', '1005'],
      'email': ['1002'],
      'phone': ['1003'],
      'company': ['1004']
    };

    const questionIds = mapping[fieldType] || [];
    const answer = answers.find(a => questionIds.includes(a.questionId));

    return answer ? answer.text : null;
  }

  // Check for duplicate lead
  async checkDuplicate(email, linkedinId) {
    const query = 'SELECT * FROM leads WHERE email = $1 OR linkedin_id = $2';
    const result = await pool.query(query, [email, linkedinId]);
    return result.rows.length > 0;
  }

  // Save lead to database
  async saveLead(leadData) {
    const query = `
      INSERT INTO leads (name, email, phone, company, linkedin_id, source, status)
      VALUES ($1, $2, $3, $4, $5, $6, $7)
      RETURNING *
    `;

    const values = [
      leadData.name,
      leadData.email,
      leadData.phone,
      leadData.company,
      leadData.linkedin_id,
      leadData.source,
      leadData.status
    ];
  }
}

```

```
    const result = await pool.query(query, values);  
    return result.rows[0];  
  }  
}  
  
module.exports = new LinkedInService();
```

3. OCR Processing Flow

Frontend Component:

```

// ocr-scanner.component.ts
export class OcrScannerComponent {
  selectedFile: File | null = null;
  extractedData: any = null;
  loading = false;

  onFileSelected(event: any) {
    this.selectedFile = event.target.files[0];
  }

  extractData() {
    if (!this.selectedFile) return;

    this.loading = true;

    const formData = new FormData();
    formData.append('file', this.selectedFile);

    this.http.post<any>('/api/ocr/extract', formData).subscribe({
      next: (response) => {
        this.extractedData = response;
        this.populateForm();
        this.loading = false;
      },
      error: (error) => {
        console.error('OCR extraction failed', error);
        this.loading = false;
      }
    });
  }

  populateForm() {
    // Auto-fill form fields with extracted data
    this.form.patchValue({
      name: this.extractedData.name,
      email: this.extractedData.email,
      phone: this.extractedData.phone,
      notes: JSON.stringify({
        university: this.extractedData.university,
        degree: this.extractedData.degree,
        skills: this.extractedData.skills
      })
    });
  }

  submitLead() {
    const leadData = this.form.value;
    this.leadService.createLead(leadData).subscribe({
      next: () => {
        // Lead created, SSE will broadcast update
        this.form.reset();
      }
    });
  }
}

```

Backend Route:

```

// ocr.js route
const multer = require('multer');
const { GoogleGenerativeAI } = require('@google/generative-ai');

const upload = multer({ storage: multer.memoryStorage() });
const genAI = new GoogleGenerativeAI(process.env.GEMINI_API_KEY);
const model = genAI.getGenerativeModel({ model: 'gemini-2.5-flash' });

router.post('/extract', upload.single('file'), async (req, res) => {
  try {
    if (!req.file) {
      return res.status(400).json({ error: 'No file provided' });
    }

    // Convert buffer to Base64
    const base64 = req.file.buffer.toString('base64');
    const mimeType = req.file.mimetype;

    // Prepare Gemini API request
    const prompt = `
Extract the following information from this document and return ONLY valid JSON:
{
  "fullName": "full name or null",
  "emailAddress": "email or null",
  "phoneNumber": "phone or null",
  "university": "university name or null",
  "degree": "degree type or null",
  "skills": "comma-separated skills or null"
}
`;

    const response = await model.generateContent([
      {
        inlineData: {
          mimeType: mimeType,
          data: base64
        }
      },
      {
        text: prompt
      }
    ]);

    // Parse response
    const text = response.response.text();
    const jsonMatch = text.match(/\{[\s\S]*\}/);

    if (!jsonMatch) {
      return res.status(400).json({ error: 'Failed to extract data' });
    }

    const extractedData = JSON.parse(jsonMatch[0]);
    res.json(extractedData);

  } catch (error) {
    console.error('OCR extraction error:', error);
    res.status(500).json({ error: error.message });
  }
});

```

ENVIRONMENT VARIABLES

Create a `.env` file in the Backend directory with the following configuration:

```
# Server Configuration
PORT=3000

# PostgreSQL Database
DB_HOST=localhost
DB_PORT=5432
DB_NAME=interndb
DB_USER=postgres
DB_PASSWORD=<your_secure_password>

# LinkedIn Webhook Security
LINKEDIN_WEBHOOK_SECRET=tankhapay_webhook_secret_2024

# Google Gemini API
GEMINI_API_KEY=<your_google_gemini_api_key>
```

Important Security Notes:

- Never commit `.env` file to version control
- Use strong, unique passwords for database
- Rotate API keys regularly
- Limit API key permissions to minimum required
- Use environment-specific configurations for development, staging, and production

DEPLOYMENT & RUNNING

Prerequisites

- **Node.js** v18+ and npm v9+
- **PostgreSQL** v12+ (running on localhost:5432)
- **Google Gemini API Key** (free tier available)
- **LinkedIn Webhook Credentials** (if using LinkedIn integration)

Local Development Setup

1. Database Setup

```
# Connect to PostgreSQL
psql -U postgres

# Create database
CREATE DATABASE interndb;

# Exit psql
\q
```

2. Backend Setup

```
cd Backend

# Install dependencies
npm install

# Create .env file with required variables
cp .env.example .env
# Edit .env with your configuration

# Start development server (auto-reload on file changes)
npm run dev

# OR start production server
npm start
```

Expected Output:

```
Backend running on http://localhost:3000
Database connected
```

3. Frontend Setup

```
cd Frontend

# Install dependencies
npm install

# Start development server
ng serve

# Application available at http://localhost:4200
```

Expected Output:

```
✓ Compiled successfully
✓ Build complete. Watching for file changes...
Application bundle generated successfully.
```

4. Verify System

```
# Test API endpoints
curl http://localhost:3000/api/health

# Access frontend
open http://localhost:4200
```

Testing the Features

LinkedIn Webhook Testing

Use Postman or curl to simulate LinkedIn webhook:

```
# 1. Challenge verification (LinkedIn will call this first)
curl -X GET "http://localhost:3000/api/webhook/linkedin?challengeCode=test123"

# 2. Process webhook payload
curl -X POST "http://localhost:3000/api/webhook/linkedin" \
  -H "Content-Type: application/json" \
  -H "X-Linkedin-Secret: tankhapay_webhook_secret_2024" \
  -d '{
    "formResponse": {
      "submittedAt": "2026-05-27T10:30:00Z",
      "answers": [
        {"questionId": "1001", "text": "John Doe"},
        {"questionId": "1002", "text": "john@example.com"},
        {"questionId": "1003", "text": "+1-555-0123"},
        {"questionId": "1004", "text": "Acme Corp"}
      ]
    }
  }'
```

OCR Scanner Testing

```
# Upload a resume/business card for extraction
curl -X POST "http://localhost:3000/api/ocr/extract" \
  -F "file=@resume.pdf"
```

Real-Time Dashboard Testing

```
# In one terminal, create a lead
curl -X POST "http://localhost:3000/api/leads" \
  -H "Content-Type: application/json" \
  -d '{
    "name": "Jane Smith",
    "email": "jane@example.com",
    "company": "TechCorp",
    "source": "Manual Entry"
  }'

# In another terminal, listen to SSE stream
curl -N http://localhost:3000/api/leads/stream
```

Production Build

Frontend

```
cd Frontend

# Build optimized bundle
ng build --configuration production

# Output in dist/frontend/ directory
```

Backend


```
cd Backend
```

```
# No additional build needed, start production server
NODE_ENV=production npm start
```

Deployment Checklist

- ☐ Set production database credentials
- ☐ Update API keys and secrets
- ☐ Enable HTTPS/SSL
- ☐ Configure CORS for production domain
- ☐ Set up database backups
- ☐ Configure error logging
- ☐ Set up monitoring and alerts
- ☐ Test all webhook endpoints with production credentials
- ☐ Verify SSE connections under load
- ☐ Document deployment procedures
- ☐ Set up CI/CD pipeline

ARCHITECTURAL DECISIONS

1. SSE over WebSocket

Decision: Use Server-Sent Events (SSE) instead of WebSocket for real-time updates.

Rationale:

- Simpler implementation and configuration
- One-way communication sufficient (server → client only)
- Better browser compatibility without additional libraries
- Lower resource overhead compared to WebSocket
- Native JavaScript `EventSource` API

Trade-offs:

- Cannot send data from client to server via SSE
- Single connection per client (WebSocket allows multiple)
- Slightly higher latency than WebSocket

2. Google Gemini AI Integration

Decision: Use Google Gemini 2.5 Flash for OCR document extraction.

Rationale:

- No infrastructure setup required (API-based)
- Cost-effective with free tier availability
- Handles multiple document types (images, PDFs)
- JSON-compatible response format
- Fast inference time

Trade-offs:

- Dependency on external API
- API rate limits and costs
- Requires internet connectivity

3. Connection Pooling with pg Library

Decision: Implement database connection pooling using native `pg` library.

Rationale:

- Efficient resource utilization
- Reuses connections instead of creating new ones
- Improved response times
- Built-in error handling and recovery
- No additional ORM dependency

Trade-offs:

- Lower-level abstraction (manual query writing)
- More SQL injection vulnerability if not careful
- Query building can be verbose

4. Modular Route Structure

Decision: Separate each feature into its own route file.

Rationale:

- Clear separation of concerns
- Easy to locate and modify features
- Simplified testing and maintenance
- Scales well as features grow
- Facilitates team collaboration

Trade-offs:

- More files to manage
- Potential code duplication across routes
- Requires consistent naming conventions

5. In-Memory File Storage with Multer

Decision: Use `multer.memoryStorage()` for file uploads.

Rationale:

- No disk I/O overhead
- Temporary processing only (not persisting files)
- Faster API processing
- Automatic memory cleanup

Trade-offs:

- Limited by available RAM
- Not suitable for large batch processing
- No persistent file storage

6. Angular State Management with Service

Decision: Use simple `AppStateService` for state management.

Rationale:

- No additional dependencies (no Redux, NgRx)
- Sufficient for application complexity
- RxJS Subjects provide reactive updates
- Easy to understand and maintain

Trade-offs:

- Not suitable for very complex state

- Limited debugging tools vs Redux
- State mutations not immutable by default

7. SQL Query Building

Decision: Write raw SQL queries instead of using ORM.

Rationale:

- Full control over query optimization
- No ORM overhead
- Easier to debug database issues
- Better performance for simple queries

Trade-offs:

- Requires SQL knowledge
- Vulnerable to SQL injection if not careful
- More verbose code
- Difficult migrations

SUMMARY

This **Internship Project** is a comprehensive, production-ready CRM system featuring:

- ❑ **Real-time Dashboard** with SSE live updates
- ❑ **LinkedIn Webhook Integration** for automated lead capture
- ❑ **AI-Powered OCR Scanner** with Google Gemini
- ❑ **Bill & Invoice Processing** with annotation tools
- ❑ **Full Authentication System** with user profiles
- ❑ **RESTful API Architecture** with modular routes
- ❑ **PostgreSQL Database** with connection pooling
- ❑ **Modern Angular 17 Frontend** with responsive design
- ❑ **Scalable Backend** with Express.js

The system demonstrates best practices in full-stack development, real-time data handling, AI integration, and modern web architecture.

Last Updated: May 27, 2026 **Version:** 1.0 **Status:** Active Development